Welcome to Code Crunch!

C3 | CRUNCH LABS: PORTFOLIO

MON 2-3PM Weekly | Spring 2025

Attendance

Earn a Certificate of Completion by attending at least 70% of the workshops

linktr.ee/CODE.CRUNCH

Unit #1

C3 | CRUNCH LABS: Java Fundamentals

What is Java?

- Java is a high-level object-oriented programming language.

- Key Features:
  - Platform-independent
  - Strong memory management
  - Robust security features.

Image taken from Oracle.

Core Java Concepts

Java Data Types:

- *Primitive*: Predefined by Java and represent simple values. Ex: int, double, char, boolean.
- *Reference*: Objects or arrays. Used to store memory addresses to the actual data.

Example of Declaring and Using Primitive Data Types:

java

Copy code

```java
public class Main {
    public static void main(String[] args) {
        // Declaring primitive data types
        int age = 25;
        double price = 19.99;
        boolean isJavaFun = true;
        char grade = 'A';

        // Using variables
        System.out.println("Age: " + age);
        System.out.println("Price: " + price);
        System.out.println("Is Java fun? " + isJavaFun);
        System.out.println("Grade: " + grade);
    }
}
```

Java Core Concepts

Example of Declaring and Using Reference Data Types:

Using Objects:

java Copy code

```java
public class Car {
    String brand;
    int year;

    public Car(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    public void displayInfo() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }
}

public class Main {
    public static void main(String[] args) {
        // Declaring and using an object (reference data type)
        Car myCar = new Car("Toyota", 2020);
        myCar.displayInfo();
    }
}
```

Using Arrays:

java Copy code

```java
public class Main {
    public static void main(String[] args) {
        // Declaring an array of integers
        int[] numbers = {1, 2, 3, 4, 5};

        // Accessing array elements
        for (int i = 0; i < numbers.length; i++) {
            System.out.println("Number at index " + i + ": " + numbers[i]);
        }
    }
}
```

Java Core Concepts

Loops:

- *For:* Used when the exact number of iterations is known beforehand.
- *While:* Used when the number of iterations is unknown, but there's a condition to control the loop.
- *Do-while:* Similar to the while loop, but it's executed at least once.

Java

```java
for (int i = 0; i < 5; i++) {
    System.out.println("Iteration: " + i);
}
```

Java

```java
int count = 0;
while (count < 10) {
    System.out.println("Count: " + count);
    count++;
}
```

Java

```java
int x = 1;
do {
    System.out.println("Value of x: " + x);
    x++;
} while (x <= 5);
```

Java Core Concepts

Conditional Statements and Operators:

Conditionals: If, if-else, if-else if-else, switch.

Operators: +, -, *, /, %, =, &&, ||, !, ==, <=, >=, !=, <, >, ^, ++, --.

Java

```java
int score = 85;

if (score >= 90) {
    System.out.println("Excellent!");
} else if (score >= 80) {
    System.out.println("Good!");
} else if (score >= 70) {
    System.out.println("Average!");
} else {
    System.out.println("Needs Improvement!");
}
```

Java

```java
int dayOfWeek = 1;

switch (dayOfWeek) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    // ... other days of the week
    default:
        System.out.println("Invalid day.");
}
```

Intro to Object-Oriented Programming (OOP)

Key Concepts:

- Programming approach that is based on the concept of “objects”. These objects represent real-world entities that combine both data (attributes) and behavior (methods).
- **Attributes**: Information or characteristics of the object.
- **Methods**: Actions the object can perform.
- **Classes**: Blueprints or templates for creating objects.
- **Objects**: Instances of classes.

Intro to Object-Oriented Programming (OOP)

OOP Principles:

- *Encapsulation:* Data and methods are bundled together inside the object. This protects data from unauthorized access and modifications.
- *Abstraction:* Hides complexity and shows only the necessary details.
- *Inheritance:* Create new classes (child) from existing ones (parent). Child classes inherit attributes and methods from parent class.
- *Polymorphism:* The same method can behave differently depending on the object calling it.

Intro to Object-Oriented Programming (OOP)

OOP Principles Examples:

- *Encapsulation:* A Car object contains both the color (attribute) of the car and the method to drive the car.
- *Abstraction:* When you drive a car, you don't need to know exactly how the engine works. You just need to know how to press the pedal to accelerate.
- *Inheritance:* A SportsCar class can inherit all the attributes and methods from the Car class, but can also have additional features like turboBoost().
- *Polymorphism:* A drive() method might work differently for a Car (driving on roads) and a Boat (driving on water).

Intro to Object-Oriented Programming (OOP)

Benefits of OOP:

- It helps to keep things organized and secure. The programmer can control how data is accessed and modified.
- It makes the system easier to use and understand by focusing on high-level operations.
- It promotes reusability and helps avoid repeating code.
- It allows flexibility and the ability to work with different types of objects using a single method or interface.
- The code can be broken into small and manageable pieces.
- It’s easier to fix or update parts of the program without affecting other parts.
- The programmer can add more without breaking the whole system.

Q&A Session

Attendance

Join us for the upcoming weekly sessions this Spring 2025!

Monday: 1PM & 2PM
Tuesdays: 1PM
Wednesdays: 2PM
Thursdays: 1PM , 2PM & 3PM
Friday: 2PM & 3PM

RSVP LU.MA/CODECRUNCH

Your feedback helps us improve. Share your thoughts!
Go to our website: go.fiu.edu/codecrunch